



Đại học Khoa học Tự nhiên

Giải thuật di truyền trong bài toán
người du lịch

Sinh viên :

Nguyễn Hữu Dũng - 23000104

Giảng viên :

LÊ HUY HOÀNG

Ngày 29 tháng 9 năm 2025

Mục lục

Danh mục ký hiệu	4
Lời nói đầu	5
1 Bài toán người du lịch (TSP - Travelling Salesman Problem)	6
1.1 Lịch sử hình thành bài toán	6
1.2 Phát biểu bài toán	7
1.3 Phân tích độ phức tạp bài toán	7
1.4 Ứng dụng thực tiễn của bài toán người du lịch	7
2 Giải thuật di truyền (GA - Genetic Algorithms)	8
2.1 Các thành phần chính	9
2.2 Nguyên lý hoạt động	9
2.3 Các toán tử di truyền	11
2.3.1 Toán tử lai ghép	11
2.3.2 Toán tử đột biến	12
2.3.3 Toán tử chọn lọc	13
2.3.4 Một số toán tử khác	14
2.4 Một số biến thể của thuật toán di truyền	15
2.4.1 Thuật toán di truyền mã hóa thực và nhị phân	15
2.4.2 Thuật toán di truyền đa mục tiêu	16
2.4.3 Thuật toán di truyền song song	16
2.4.4 Thuật toán di truyền hỗn loạn	16
2.4.5 Thuật toán di truyền lai ghép	17
3 Thực nghiệm	17
3.1 Bộ dữ liệu	17
3.2 Các bước xây dựng giải thuật GA	18
4 Đánh giá và kết luận	22
4.1 So sánh hiệu quả giữa các phép lai ghép	23
4.2 Phương án cải thiện đề xuất: kết hợp với tìm kiếm Tabu	24

Lời cảm ơn

Đầu tiên, chúng em muốn bày tỏ lòng biết ơn sâu sắc đến *Trường Đại học Khoa học Tự nhiên, Đại học quốc gia Hà Nội* và đặc biệt là các thầy cô giáo trong khoa *Toán Cơ Tin học*. Trong suốt thời gian học vừa qua, sự hướng dẫn và kiến thức quý báu mà các thầy cô đã truyền đạt đã làm cho chúng em trưởng thành hơn về chuyên môn, về nhân cách và về đạo đức. Chúng em xin chân thành cảm ơn thầy giáo, *TS Lê Huy Hoàng*, người đã tận tình hướng dẫn, giúp đỡ chúng em trong suốt quá trình thực hiện và hoàn thành bài nghiên cứu này. Thầy không chỉ cung cấp tài liệu mà còn truyền đạt những phương pháp và định hướng nghiên cứu quan trọng. Chúng em cũng muốn gửi lời cảm ơn chân thành đến gia đình và bạn bè, những người đã luôn ủng hộ, động viên và giúp đỡ em trong quá trình học tập và nghiên cứu. Cuối cùng, chúng em trân trọng biết ơn và đánh giá cao những kinh nghiệm quý báu mà chúng em đã được học từ các thầy cô và những người xung quanh. Những kiến thức và phẩm chất mà em đã tích lũy trong thời gian học tập này sẽ luôn là tài sản quan trọng và là nguồn động lực để em tiếp tục phát triển trong cuộc sống và sự nghiệp.

Danh mục ký hiệu

TSP	Bài toán người du lịch.
GA	Giải thuật di truyền.
GAs	Các giải thuật di truyền (viết tắt của Genetic Algorithms).
MOGA	Giải thuật di truyền đa mục tiêu
NST	Nhiễm sắc thể, tượng trưng cho một giải pháp của bài toán.
Schemas	Cấu trúc trong GA biểu diễn thông tin giải pháp.
Population	Quần thể, tập hợp các nhiễm sắc thể trong thể hệ.
Fitness	Hàm thích nghi, hàm đánh giá chất lượng lời giải.

Lời nói đầu

Bài toán người du lịch (Traveling Salesman Problem - TSP) là một trong những bài toán tối ưu tổ hợp cổ điển nhất trong lĩnh vực khoa học máy tính và toán học ứng dụng. Với tính chất phức tạp và đa dạng, bài toán này đã thu hút sự quan tâm rộng rãi từ cộng đồng nghiên cứu, đặc biệt trong bối cảnh ứng dụng thực tiễn ngày càng phong phú như lập kế hoạch lộ trình giao hàng, thiết kế mạng lưới giao thông, và tối ưu hóa quy trình sản xuất.

Giải quyết bài toán TSP không chỉ mang ý nghĩa lý thuyết mà còn đóng góp quan trọng vào việc nâng cao hiệu quả vận hành trong nhiều lĩnh vực. Trong báo cáo này, thuật toán di truyền – một phương pháp tối ưu dựa trên nguyên lý chọn lọc tự nhiên – được đề xuất như một giải pháp hiệu quả để tiếp cận bài toán này.

Mục tiêu của nghiên cứu là ứng dụng thuật toán di truyền để giải quyết bài toán TSP, đồng thời đánh giá khả năng cải tiến hiệu suất tính toán và chất lượng giải pháp. Thông qua đó, nghiên cứu kỳ vọng phát triển một phương pháp hiệu quả và tính ứng dụng cao, góp phần vào việc mở rộng kiến thức trong lĩnh vực tối ưu hóa và khoa học dữ liệu.

Phạm vi nghiên cứu bao gồm việc nghiên cứu đề xuất, thử nghiệm và đánh giá hiệu quả của thuật toán di truyền trong việc giải quyết bài toán TSP. Nghiên cứu sẽ tập trung vào các bộ dữ liệu TSP có quy mô vừa và lớn, đồng thời đầu tư nghiên cứu vào việc cải tiến tham số để tăng hiệu suất tính toán và chất lượng giải pháp.

Báo cáo tập trung phân tích các khía cạnh lý thuyết và thực nghiệm, bao gồm: tổng quan về bài toán TSP và các thuật toán giải, nguyên lý áp dụng thuật toán di truyền, thử nghiệm trên các bộ dữ liệu chuẩn, và đề xuất cải tiến thuật toán nhằm tăng cường hiệu quả. Kết quả nghiên cứu hứa hẹn mang lại những đóng góp thiết thực, không chỉ trong lý thuyết mà còn trong ứng dụng thực tiễn.

1 Bài toán người du lịch (TSP - Travelling Salesman Problem)

1.1 Lịch sử hình thành bài toán

Bài toán TSP được định nghĩa trong thế kỉ 19 bởi nhà toán học Ireland William Rowan Hamilton và nhà toán học Anh Thomas Kirkman. Trường hợp tổng quát của TSP có thể được nghiên cứu lần đầu tiên bởi các nhà toán học ở Vienna và Harvard trong những năm 1930 và là một trong những bài toán được nghiên cứu sâu nhất trong tối ưu hóa. Nó thường được dùng làm thước đo cho nhiều phương pháp tối ưu hóa. Mặc dù bài toán rất khó giải trong trường hợp tổng quát, có nhiều phương pháp giải chính xác cũng như heuristic đã được tìm ra để giải quyết một số trường hợp có tới hàng chục nghìn thành phố.

Nguồn gốc của bài toán này vẫn chưa được làm rõ, tuy nhiên, một cuốn sổ tay của người bán hàng xuất bản năm 1832 có đề cập tới bài toán này với ví dụ cho chu trình nước Đức và Thụy Sĩ, dù bên trong không có bất cứ công thức toán học nào.

Trong những năm 1950 và 1960, bài toán trở nên phổ biến trong giới nghiên cứu khoa học ở Châu Âu và Mỹ. Trong đó, George Dantzig, Delbert Ray Fulkerson và Selmer M. Johnson ở công ty RAND tại Santa Monica đã có đóng góp quan trọng cho bài toán này, bằng cách biểu diễn bài toán dưới dạng quy hoạch nguyên và đưa ra phương pháp mặt phẳng cắt để tìm ra lời giải. Với phương pháp mới này, họ đã giải được tối ưu một trường hợp có 49 thành phố bằng cách xây dựng một chu trình và chứng minh rằng không có chu trình nào ngắn hơn. Trong những thập niên tiếp theo, bài toán được nghiên cứu bởi nhiều nhà nghiên cứu trong các lĩnh vực toán học, khoa học máy tính, hóa học, vật lý, và các ngành khác.

Năm 1972, Richard M. Karp chứng minh rằng bài toán chu trình Hamilton là NP-đầy đủ, kéo theo bài toán TSP cũng là NP-đầy đủ. Đây là một lý giải toán học cho sự khó khăn trong việc tìm kiếm chu trình ngắn nhất.

Một bước tiến lớn được thực hiện cuối thập niên 1970 và 1980 khi Grötschel, Padberg, Rinaldi và cộng sự đã giải được những trường hợp lên tới 2392 thành phố, sử dụng phương pháp mặt phẳng cắt và nhánh cận. Và cho đến nay, bài toán TSP vẫn được nghiên cứu để tìm lời giải cho các bộ dữ liệu lớn hơn.

1.2 Phát biểu bài toán

Bài toán người du lịch là một trong những bài toán tối ưu hóa nổi tiếng nhất trong lĩnh vực khoa học máy tính và nghiên cứu hoạt động. Bài toán yêu cầu tìm một hành trình ngắn nhất hoặc một chi phí nhỏ nhất để một người bán hàng đi qua tất cả các thành phố trong một tập các thành phố đã cho sao cho mỗi thành phố chỉ đi qua đúng một lần và trở về thành phố xuất phát ban đầu.

Phát biểu bài toán TSP như sau: Cho một đồ thị có trọng số đầy đủ $G = (V, E)$, trong đó V là tập hợp các thành phố và E là tập hợp các đường đi giữa các thành phố, mỗi đường đi có một trọng số không âm. Yêu cầu là tìm một chu trình Hamilton (một hành trình qua tất cả các thành phố một lần và trở về thành phố ban đầu) có tổng trọng số là nhỏ nhất.

1.3 Phân tích độ phức tạp bài toán

Độ phức tạp của bài toán TSP được xếp vào loại bài toán NP-đầy đủ (NP-complete), một trong những lớp bài toán khó nhất trong lý thuyết độ phức tạp tính toán.

TSP yêu cầu tìm một chu trình tối thiểu qua n thành phố sao cho mỗi thành phố được đi qua đúng một lần và trở về điểm xuất phát với tổng khoảng cách là nhỏ nhất. Với n thành phố, có tổng cộng $\frac{(n-1)!}{2}$ chu trình khác nhau cần phải xem xét nếu sử dụng phương pháp liệt kê tất cả các khả năng, dẫn đến độ phức tạp thời gian của bài toán theo hàm giai thừa.

Không có thuật toán nào được biết đến có thể giải bài toán TSP trong thời gian đa thức cho mọi trường hợp và bất kỳ giải pháp chính xác nào cho TSP cũng đòi hỏi thời gian tính toán tăng theo cấp số nhân với số lượng thành phố. Mặc dù không có lời giải chính xác nhanh chóng cho TSP, nhiều phương pháp gần đúng đã được phát triển để đưa ra lời giải tốt trong thời gian hợp lý như: một số thuật toán Heuristic, thuật toán nhánh cận và mặt phẳng cắt, thuật toán Tabu,...

1.4 Ứng dụng thực tiễn của bài toán người du lịch

Phần lớn công trình nghiên cứu về bài toán người du lịch với mục đích làm nền tảng nghiên cứu các phương pháp chung có thể áp dụng cho nhiều vấn đề tối ưu hóa rời rạc. Thật vậy, nhiều ứng dụng trực tiếp của TSP mang lại giá trị cho lĩnh vực nghiên cứu như

làm bài toán phụ để giải bài toán khó, trong học máy, phân tích Gen trong nghiên cứu sinh học,...

Ngoài ra, bài toán TSP có nhiều ứng dụng trong thực tế:

- Vận chuyển và hậu cần: trong lĩnh vực vận chuyển hàng hóa, bài toán TSP thường được sử dụng để tối ưu hóa hành trình của các xe vận chuyển. Bằng cách xác định một hành trình ngắn nhất để ghé thăm các điểm đến, doanh nghiệp có thể giảm thiểu chi phí vận chuyển và tăng hiệu quả hoạt động.
- Lập kế hoạch và lập lịch: bài toán TSP cũng được áp dụng trong lập kế hoạch và lập lịch công việc. Ví dụ, trong việc lập lịch công việc cho các kỹ sư hoặc kỹ thuật viên, bài toán TSP có thể giúp xác định một hành trình tối ưu để họ thăm các địa điểm khác nhau trong một khu vực nhất định.
- Định tuyến mạng: trong các mạng máy tính và mạng viễn thông, bài toán TSP được áp dụng để tối ưu hóa định tuyến dữ liệu. Bằng cách xác định hành trình ngắn nhất để truyền dữ liệu qua các nút mạng, hệ thống mạng có thể hoạt động hiệu quả hơn và giảm thiểu độ trễ.
- Tính toán tối ưu: bài toán TSP cũng có ứng dụng trong nhiều lĩnh vực tính toán tối ưu khác, bao gồm thiết kế mạch tích hợp, quy hoạch sản xuất, lập lịch sản xuất và tối ưu hóa các quá trình công nghiệp. Bằng cách tối ưu hóa hành trình hoặc sắp xếp các công việc, bài toán TSP có thể cải thiện hiệu quả và giảm thiểu chi phí trong quá trình này.

2 Giải thuật di truyền (GA - Genetic Algorithms)

Hiện tại vẫn chưa có thuật toán nào giải quyết chính xác tổng quát nhất các dạng bài toán NP-Hard trong thời gian đa thức. Vì vậy, các nhà nghiên cứu tìm đến các phương pháp xấp xỉ (heuristic và meta-heuristic) để tìm ra các lời giải gần tối ưu trong thời gian chấp nhận được. Các phương pháp này không đảm bảo tìm ra lời giải tốt nhất, nhưng thường tìm thấy các lời giải đủ tốt một cách nhanh chóng và dễ dàng. Thuật toán di truyền là một thuật toán tối ưu hoá trong lớp thuật toán MetaHeuristic, bao gồm những thuật toán được lấy ý tưởng từ tự nhiên như quá trình tiến hoá sinh học, tập tính bầy

dàn hay định luật trong vật lý. J.H. Holland đã đề xuất GA vào năm 1992, lấy cảm hứng từ quá trình chọn lọc tự nhiên.

2.1 Các thành phần chính

GA được lấy cảm hứng từ quá trình tiến hóa tự nhiên, đặc biệt là thuyết "chọn lọc tự nhiên" của Darwin, trong đó những cá thể khỏe mạnh (có độ thích nghi cao) có khả năng sống sót và sinh sản cao hơn. GA mô phỏng quá trình này để tìm kiếm giải pháp tối ưu cho một vấn đề cụ thể.

Thuật toán di truyền sử dụng một số thành phần chính:

- **Nhiễm sắc thể (NST):** Đại diện cho một giải pháp tiềm năng cho bài toán. Nhiễm sắc thể thường được mã hóa dưới dạng chuỗi nhị phân hoặc thứ tự. Các nhiễm sắc thể này có thể được xem như các điểm (cá thể) trong không gian nghiệm (quần thể) của bài toán.
- **Quần thể (Population):** Một tập hợp các nhiễm sắc thể (cá thể), đại diện cho một tập các giải pháp tiềm năng. Quần thể ban đầu thường được tạo ra một cách ngẫu nhiên.
- **Hàm thích nghi (Fitness function):** Được sử dụng để đánh giá độ tốt của mỗi nhiễm sắc thể trong quần thể. Hàm này gán một giá trị cho mỗi nhiễm sắc thể, cho biết mức độ phù hợp của nó với bài toán. Các nhiễm sắc thể có giá trị fitness cao hơn sẽ được ưu tiên trong các bước tiếp theo.
- **Các toán tử di truyền (Genetic operators):** Các thao tác được sử dụng để tạo ra các nhiễm sắc thể mới từ các nhiễm sắc thể hiện tại, bao gồm: chọn lọc, lai ghép, đột biến và mã hóa.

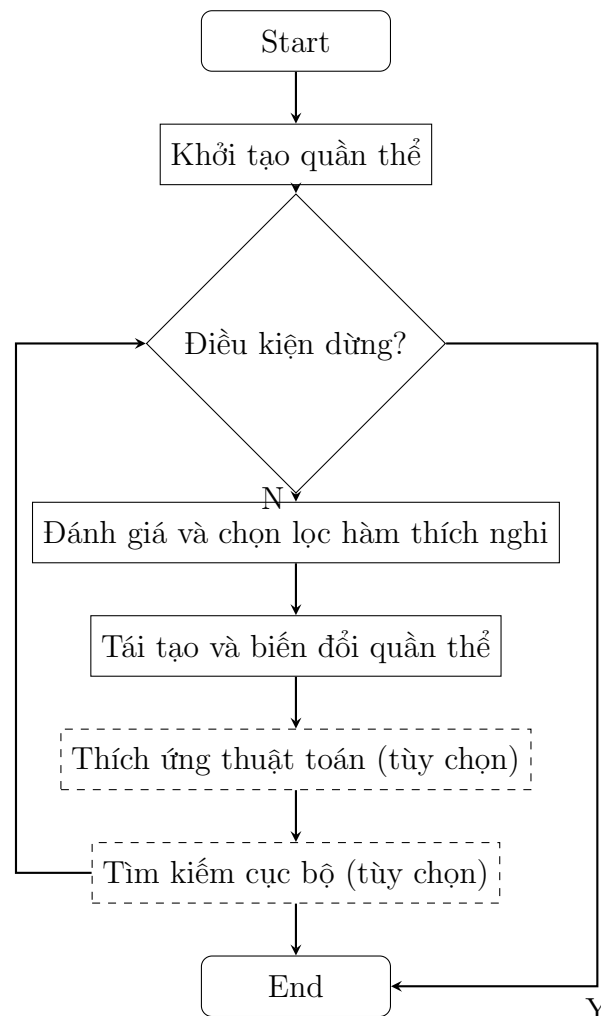
2.2 Nguyên lý hoạt động

Thuật toán chung của một giải thuật di truyền theo các bước [1]:

- + Chọn một quần thể ban đầu $x_0 = \{x_{10}, x_{20}, \dots, x_{N0}\}$, $x_i \in S$, trong đó S là không gian tìm kiếm;
- + Xác định giá trị hàm mục tiêu $f(x_0)$ cho mỗi cá thể của quần thể;

- + Lặp lại cho mọi lần lặp j cho đến khi đạt điều kiện dừng;
- + Thực hiện chọn lọc;
- + Thực hiện lai ghép với một xác suất;
- + Thực hiện đột biến với một xác suất;
- + Xác định quần thể mới x_i và hàm fitness f_i ;
- + Thay thế nếu các cá thể mới có fitness tốt hơn;
- + Ngược lại, giữ nguyên các cá thể và tiếp tục với các lần lặp tiếp theo.

Thuật toán lặp lại quá trình đánh giá, chọn lọc, lai ghép, và đột biến cho đến khi đạt một trong các điều kiện dừng: đạt số thế hệ tối đa, giá trị fitness tối ưu hoặc không có sự cải thiện đáng kể sau một số thế hệ.



2.3 Các toán tử di truyền

Người ta thường sử dụng toán tử mã hoá (Encoding) để chuyển đổi các thông tin của bài toán thành dạng chuỗi bit hoặc các cấu trúc dữ liệu khác để GA có thể xử lý. Các phương pháp mã hóa bao gồm mã hóa nhị phân, mã hóa số thực, mã hóa hoán vị, mã hóa cây. Ngoài ra, dạng thuật toán di truyền đơn giản nhất bao gồm ba loại toán tử: chọn lọc, lai ghép (tái tổ hợp) và đột biến [2].

2.3.1 Toán tử lai ghép

Lai ghép (tái tổ hợp) là một đặc điểm chính của thuật toán di truyền (GA). Lai ghép (crossover operators) trong thuật toán di truyền (GA) là các phương pháp kết hợp thông tin di truyền từ hai hoặc nhiều cá thể cha mẹ để tạo ra các cá thể con mới. Mục đích của việc này là để khám phá không gian tìm kiếm và tạo ra các giải pháp tốt hơn.

Hai cá thể được chọn bằng toán tử chọn lọc và các vị trí lai ghép được chọn ngẫu nhiên. Sau đó, các gen tại các vị trí lai ghép này được trao đổi, do đó tạo ra một cá thể hoàn toàn mới (con cái). Quá trình lai ghép giúp tăng tính đa dạng của quần thể và khám phá các vùng không gian tìm kiếm mới, từ đó giúp thuật toán tránh bị mắc kẹt vào các cực trị cục bộ.

Một số phép lai ghép (tái tổ hợp) thường gặp:

- Lai ghép một điểm cắt (Single-point crossover): Chọn một điểm lai ghép ngẫu nhiên trên chuỗi NST của hai cá thể cha mẹ và trao đổi các đoạn NST sau điểm lai ghép giữa hai cha mẹ để tạo ra hai con. Ví dụ: với 2 NST cha mẹ [1, 2, 3, 4, 5, 6, 7, 8] và [5, 6, 7, 8, 1, 2, 3, 4], chọn điểm cắt sau vị trí thứ 4, tiến hành lai chéo ta được 2 NST con lần lượt là: [1, 2, 3, 4, 5, 6, 7, 8], [5, 6, 7, 8, 1, 2, 3, 4].
- Lai ghép hai điểm cắt (Two-point crossover): Tương tự như lai ghép một điểm, nhưng chọn hai điểm lai ghép và trao đổi đoạn NST giữa hai điểm này. Giả sử hai cá thể cha mẹ trong bài toán TSP: [1, 2, 3, 4, 5, 6, 7, 8] và [5, 6, 7, 8, 1, 2, 3, 4]. Chọn hai điểm cắt tại vị trí 3 và 6:

[1, 2] | [7, 8, 1, 2] | [7, 8]. Loại bỏ trùng lặp, ta được: [1, 2, 7, 8, 3, 4, 5, 6].

[5, 6] | [3, 4, 5, 6] | [3, 4]. Loại bỏ trùng lặp, ta được: [5, 6, 3, 4, 7, 8, 1, 2].

- Lai ghép đa điểm cắt (k-point crossover): ở rộng của lai ghép hai điểm, sử dụng k điểm lai ghép để trao đổi các đoạn NST giữa các điểm.
- Lai ghép đồng nhất (Uniform crossover): Mỗi gen trong NST con được chọn từ một trong hai gen tương ứng của cha mẹ một cách ngẫu nhiên.
- Lai ghép có thứ tự (Order crossover): Chọn một tập hợp các gen theo thứ tự từ một parent, và giữ nguyên thứ tự của các gen đó, sau đó dùng các gen từ parent còn lại để điền vào các chỗ còn trống.
- Lai ghép đối xứng từng phần (Partially matched crossover - PMX): thường sử dụng cho các bài toán hoán vị. Phép lai tạo ra một ánh xạ giữa các gen của hai cha mẹ, sau đó sử dụng ánh xạ này để tạo ra các con. PMX thường được dùng với mã hóa hoán vị và đột biến đảo vị để cho ra kết quả tối ưu.
- Lai ghép số học (Arithmetic crossover): phép lai tạo ra các con bằng cách kết hợp tuyến tính các giá trị gen của cha mẹ. Ví dụ: $con = \alpha * cha + (1 - \alpha) * mẹ$, với α là một số ngẫu nhiên trong khoảng (0,1).
- Lai ghép chu kỳ (Cycle crossover - CX): phép lai tạo ra các con bằng cách duy trì các chu kỳ gen từ cha mẹ.
- Lai ghép "tham số hóa đồng nhất" (parameterized uniform crossover): trong đó mỗi vị trí bit có xác suất p (thường từ 0.5 đến 0.8) để được hoán đổi. Phương pháp này không có ưu tiên vị trí, nhưng có thể làm gián đoạn các tổ hợp allele đã thích nghi.
- Một số phép lai ghép khác như: lai ghép xáo trộn (shuffle crossover), lai ghép bảo toàn thứ tự (Precedence preserving crossover),...

Việc chọn loại tái tổ hợp phù hợp phụ thuộc vào hàm thích nghi. Hiện tại, hai phương pháp phổ biến nhất là tái tổ hợp hai điểm và tái tổ hợp tham số hóa đồng nhất.

2.3.2 Toán tử đột biến

Trong các bài toán GA, đột biến là một trong những toán tử quan trọng nhất, sử dụng phương pháp thay đổi ngẫu nhiên một hoặc nhiều gen trong NST của một cá thể, nhằm tăng tính đa dạng của quần thể và giúp GA tránh bị mắc kẹt vào các cực trị cục

bộ giúp duy trì tính đa dạng của quần thể. Nếu không có đột biến, quần thể có thể trở nên quá đồng nhất và mất khả năng khám phá các vùng không gian tìm kiếm mới.

Một số dạng đột biến thường gặp:

- Đột biến bit (Bit flip mutation): đảo giá trị của một hoặc nhiều bit ngẫu nhiên trong NST. Ví dụ, nếu một bit có giá trị là 0, nó sẽ được đổi thành 1, và ngược lại. Dạng này thường được sử dụng cho các bài toán sử dụng mã hóa nhị phân.
- Đột biến đảo vị (Inversion mutation): Đảo ngược một chuỗi con ngẫu nhiên trong NST. Ví dụ, giả sử với NST con ban đầu là [1, 2, 3, 4, 5, 6, 7, 8], sau khi đột biến (đảo ngược chuỗi [4,5,6,7]): ta được NST mới: [1, 2, 3, 7, 6, 5, 4, 8].
- Đột biến hoán đổi (Swap mutation): Chọn ngẫu nhiên hai gen trong NST và hoán đổi vị trí của chúng. Ví dụ, giả sử với NST con ban đầu là [1, 2, 3, 4, 5, 6, 7, 8], sau khi đột biến (hoán đổi phần tử ở vị trí 2 và 6): ta được NST mới: [1, 6, 3, 4, 5, 2, 7, 8].
- Đột biến xáo trộn (Scramble mutation): Xáo trộn ngẫu nhiên các gen trong một chuỗi con của NST và kiểm tra xem liệu giải pháp mới tạo ra có cải thiện giá trị fitness hay không.

Một tham số quan trọng trong phép đột biến là xác suất đột biến (mutation rate). Nếu xác suất đột biến quá cao, GA có thể trở thành một thuật toán tìm kiếm ngẫu nhiên. Nếu xác suất đột biến quá thấp, GA có thể hội tụ quá nhanh và bỏ qua các giải pháp tốt. Thông thường, xác suất đột biến được đặt ở mức thấp, chẳng hạn như 0.01 hoặc 0.05. Không có toán tử đột biến nào là tốt nhất cho mọi bài toán. Việc lựa chọn toán tử đột biến phù hợp thường phụ thuộc vào đặc điểm của bài toán, cách mã hóa và các toán tử khác trong GA.

2.3.3 Toán tử chọn lọc

Ý tưởng của toán tử này là ưu tiên những cá thể có điểm thể lực tốt và cho phép chúng truyền gen của mình cho các thế hệ tiếp theo. Toán tử này chọn lọc các nhiễm sắc thể trong quần thể để sinh sản. Nhiễm sắc thể càng phù hợp thì càng có nhiều khả năng được chọn để sinh sản. Một số loại toán tử chọn lọc: [2].

- Chọn lọc xếp hạng (Rank selection): Sử dụng thứ hạng thay vì giá trị fitness để chọn các cá thể, giảm nguy cơ hội tụ sớm vào cực tiểu cục bộ.
- Chọn lọc giải đấu (Tournament selection): Các cá thể được chọn theo cặp và cá thể có fitness cao hơn được đưa vào nhóm của thế hệ tiếp theo.
- Chọn lọc ưu tú (Elitism selection): Đảm bảo rằng cá thể ưu tú trong một thế hệ luôn được truyền sang thế hệ tiếp theo. Nếu cá thể có giá trị fitness cao nhất không có mặt trong thế hệ tiếp theo sau quy trình chọn lọc thông thường, thì cá thể ưu tú cũng sẽ tự động được đưa vào thế hệ tiếp theo.
- Chọn lọc Boltzmann (Boltzmann selection): dựa trên phương pháp lấy mẫu và entropy, được sử dụng trong Mô phỏng Monte Carlo. Xác suất chọn được chuỗi tốt nhất là rất cao trong khi nó thực thi trong thời gian rất ngắn. Đây là phương pháp tốt để giảm nguy cơ mắc kẹt tại cực trị địa phương.

Tốc độ hội tụ của thuật toán di truyền phụ thuộc vào áp lực chọn lọc. Nếu áp lực chọn lọc quá cao, quần thể có thể mất tính đa dạng, dẫn đến hội tụ sớm, tức là thuật toán có thể bị mắc kẹt ở một giải pháp không tối ưu. Do đó, việc kiểm soát áp lực chọn lọc là rất quan trọng để đảm bảo rằng GA có thể tìm được giải pháp tối ưu toàn cục.

2.3.4 Một số toán tử khác

Ngoài tái tổ hợp và đột biến, còn nhiều toán tử khác đã được đề xuất để duy trì sự đa dạng trong quần thể, như:

- Crowding: Cá thể con thay thế cá thể trong quần thể gần giống nhất với nó.
- Fitness Sharing: Fitness của mỗi cá thể giảm dần dựa trên sự hiện diện của các cá thể khác có độ tương đồng cao.
- Hạn chế ghép đôi: Chỉ những cá thể đủ khác biệt hoặc đủ giống nhau mới được phép ghép đôi, để thúc đẩy sự hình thành các "loài" hoặc duy trì sự đa dạng.
- Mating Tags: Các phần của nhiễm sắc thể được sử dụng để nhận diện bạn đời tiềm năng. Chỉ những cá thể có tag tương ứng mới được phép ghép đôi.

- Một chiến lược khác là hạn chế ghép đôi trong không gian, nơi các cá thể chỉ có thể ghép đôi với hàng xóm gần nhất. Điều này giúp duy trì các loài biệt lập và thúc đẩy đổi mới tại các ranh giới giữa các loài. [2].

2.4 Một số biến thể của thuật toán di truyền

Các biến thể của GA được phân loại rộng rãi thành năm loại chính, cụ thể là: mã hóa thực và nhị phân, đa mục tiêu, song song, hỗn loạn và lai [1].

2.4.1 Thuật toán di truyền mã hóa thực và nhị phân

Dựa trên cách biểu diễn nhiễm sắc thể, thuật toán di truyền (GA) được chia thành hai loại, cụ thể là GA mã hóa nhị phân và GA mã hóa thực.

a) Thuật toán di truyền mã hóa nhị phân

Biểu diễn nhị phân được sử dụng để mã hóa GA và được gọi là GA nhị phân, phương pháp này có độ phức tạp tính toán cao. Các toán tử di truyền cũng đã được điều chỉnh để thực hiện quá trình tìm kiếm. Mã hóa nhị phân là một trong những phương pháp mã hóa phổ biến trong GAs, đặc biệt là trong các bài toán có không gian tìm kiếm rời rạc.

Mã hóa nhị phân (tức là chuỗi bit) là mã hóa phổ biến nhất. Những lý thuyết GA hiện có được dựa trên giả định về mã hóa nhị phân có độ dài cố định và thứ tự cố định. Một số ưu và nhược điểm:

- Tốc độ hội tụ nhanh: GAs mã nhị phân thường có tốc độ hội tụ nhanh và dễ dàng đạt được hội tụ sớm, giúp tìm ra giải pháp tối ưu trong thời gian ngắn [3].
- Tìm kiếm toàn cục: Chúng có khả năng tìm kiếm toàn cục, đơn giản trong thao tác và không yêu cầu đạo hàm, phù hợp với nhiều loại bài toán tối ưu hóa [3].
- Tuy nhiên, mã hóa nhị phân không tự nhiên, khó sử dụng cho nhiều vấn đề và chúng dễ bị sắp xếp khá tùy tiện.

b) Thuật toán di truyền mã hóa thực (Real-coded GA)

Các thuật toán di truyền mã hóa thực (RGAs) đã được sử dụng rộng rãi và ứng dụng nhiều trong thực tế. Biểu diễn nhiễm sắc thể liên quan chặt chẽ với các vấn đề trong đời sống thực. Chúng được áp dụng rộng rãi trong các bài toán tối ưu hóa có không gian tìm kiếm liên tục và thường được coi là phù hợp hơn cho các ứng dụng kỹ thuật thực tế.

Ưu thế chính của RGAs là hiệu quả và chính xác. Tuy nhiên, RGAs dễ gặp phải hiện tượng hội tụ sớm. Các nhà nghiên cứu đang làm việc để cải thiện hiệu suất của RGAs. Hầu hết RGAs được phát triển bằng cách thay đổi các toán tử giao phối, đột biến và chọn lọc.

Các toán tử di truyền bao gồm lai ghép và đột biến, là những yếu tố quyết định giúp GAs mã hóa thực có thể giải quyết các vấn đề tối ưu hóa phức tạp trong thực tế, dù chúng vẫn gặp phải một số hạn chế như hội tụ sớm và dính vào cực trị cục bộ [3].

2.4.2 Thuật toán di truyền đa mục tiêu

Thuật toán di truyền đa mục tiêu (Multiobjective Genetic Algorithm - MOGA) là một biến thể của thuật toán di truyền (GA) được thiết kế để giải quyết các bài toán tối ưu hóa có nhiều mục tiêu xung đột nhau. Điểm khác biệt chính giữa MOGA và GA là cách gán hàm thích nghi (fitness function). Trong khi GA chỉ có một mục tiêu duy nhất, MOGA phải xử lý nhiều mục tiêu cùng một lúc. Các bước còn lại của MOGA tương tự như GA. Mục tiêu chính của MOGA là tạo ra tập Pareto tối ưu trong không gian mục tiêu sao cho không có bất kỳ sự cải thiện nào trong bất kỳ hàm thích nghi nào mà không làm ảnh hưởng đến các hàm thích nghi khác. Các thuật toán di truyền đa mục tiêu được chia thành hai loại chính: dựa trên Pareto và dựa trên phân rã [3].

2.4.3 Thuật toán di truyền song song

Mục đích của GA song song là cải thiện thời gian tính toán và chất lượng của các giải pháp thông qua việc phân phối các cá thể. Các thuật toán di truyền song song được phân thành ba loại chính: các thuật toán di truyền song song chủ-nô (master-slave), các thuật toán di truyền song song hạt mịn (fine grained), và các thuật toán di truyền song song hạt thô đa quần thể (multi-population coarse grained) [3].

2.4.4 Thuật toán di truyền hỗn loạn

Thuật toán di truyền hỗn loạn (Chaotic Genetic Algorithm - Chaotic GA) là một biến thể của thuật toán di truyền (GA) được thiết kế để cải thiện khả năng tìm kiếm và tránh hội tụ sớm bằng cách kết hợp các hệ thống hỗn loạn vào quá trình tiến hóa. Sự đa

dạng của GA hỗn loạn giúp loại bỏ hội tụ sớm. Các toán tử lai ghép và đột biến có thể được thay thế bằng các ánh xạ hỗn loạn (chaotic maps).

2.4.5 Thuật toán di truyền lai ghép

Các thuật toán di truyền có thể dễ dàng lai với các phương pháp tối ưu hóa khác để cải thiện hiệu suất của chúng như các phương pháp khử nhiễu hình ảnh, tối ưu hóa phản ứng hóa học, và nhiều hơn nữa. Các lợi ích chính của việc lai GA với các phương pháp khác là chất lượng giải pháp tốt hơn, hiệu quả hơn, đảm bảo các giải pháp khả thi và tối ưu hóa các tham số điều khiển [4]. Theo các nghiên cứu cho thấy khả năng lấy mẫu của GA bị ảnh hưởng nhiều bởi kích thước quần thể. Để giải quyết vấn đề này, ta hướng tới giải pháp tích hợp các thuật toán tìm kiếm cục bộ như thuật toán memetic, Baldwinian, Lamarckian, Tabu... với GA. Một vấn đề khác trong GA là tìm các tham số điều khiển (hệ số lai ghép, hệ số chọn lọc, điều kiện dừng) phù hợp. Có thể sử dụng các kỹ thuật MetaHeuristics khác với GA để giải quyết vấn đề này.

3 Thực nghiệm

3.1 Bộ dữ liệu

Tiến hành thực nghiệm trên bộ dữ liệu gồm 4 file CSV chứa tọa độ của các thành phố (x, y) do Maxwell thu thập và phát triển:

- Dữ liệu siêu nhỏ: cỡ 10 thành phố
- Dữ liệu nhỏ: cỡ 30 thành phố
- Dữ liệu vừa: cỡ 100 thành phố
- Dữ liệu lớn: cỡ 1000 thành phố

Trong lần thực nghiệm này, với bài toán sắp xếp thứ tự các thành phố trên đường đi, GA mã hoá thực (RGAs) được lựa chọn vì tính đơn giản và trực quan của phương pháp này.

3.2 Các bước xây dựng giải thuật GA

Một nhiễm sắc thể (hay một lời giải chấp nhận được) là một danh sách có thứ tự các thành phố sẽ đi qua. Ví dụ: $[0, 3, 7, 1, 5, 8, 6, 2]$.

Bước 1: Đọc dữ liệu đầu vào và tạo ma trận khoảng cách giữa các thành phố. Khoảng cách giữa 2 thành phố $A(x_A, y_A)$, $B(x_B, y_B)$ bất kỳ ở đây được tính theo công thức chuẩn Euclid:

$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

Bước 2: Định nghĩa hàm thích nghi $fitness()$ nhận đầu vào là một NST. Hàm thích nghi có giá trị càng cao thì NST đó càng "tốt".

Với NST $X = \{X_i, i = \overline{1, n}\}$, đánh giá độ thích nghi thông qua hàm thích nghi được lựa chọn như sau:

$$fitness_{NST} = \frac{1}{\sum_{i=0}^{n-1} d(X_i, X_{i+1})}$$

Bước 3: Khởi tạo quần thể.

Algorithm 1 Initialization population (Khởi tạo quần thể)

```
1: Function InitializePopulation(cities)
2: population  $\leftarrow []$ 
3: num_cities  $\leftarrow \text{length}(\text{cities})$ 
4: pop_size  $\leftarrow 100$ 
5: for each individual in range(pop_size) do
6:   individual  $\leftarrow \text{list}(\text{range}(\text{num\_cities}))$ 
7:   shuffle(individual)
8:   distance  $\leftarrow \text{fitness}(\text{individual}, \text{distance\_mat}=\text{distance\_matrix}(\text{cities}))$ 
9:   append(population, [distance, individual])
10: end for
11: return population
```

Số lượng NST càng nhiều thời gian xử lý, tính toán càng kéo dài, số lượng NST nếu quá ít dễ thiếu sự đa dạng trong quần thể, dẫn tới tình trạng mắc kẹt trong cực trị địa phương. Vì vậy, trong quá trình thực nghiệm, sau một vài phép thử với số lượng NST khác nhau, chúng tôi cố định số lượng NST trong quần thể là 100.

Bước 4: Định nghĩa phép các toán tử chọn lọc, lai ghép và đột biến.

- **Chọn lọc:** Sử dụng phương pháp chọn lọc giải đấu (Tournament Selection). Trong

giải thuật di truyền, sử dụng chọn lọc giải đấu tổng thể : bằng cách chia quần thể thành từng nhóm nhỏ và so sánh độ thích nghi của các NST trong từng nhóm, từ đó chọn ra NST "tốt nhất". NST chiến thắng trong nhóm sẽ được chọn để tiếp tục vào giai đoạn lai ghép và tạo ra thế hệ mới. Lặp lại quá trình này cho đến khi chọn đủ số cá thể cần thiết cho bước lai ghép. Đây là một phương pháp hiệu quả để chọn lọc các NST trong thuật toán di truyền, với khả năng kiểm soát áp lực chọn lọc thông qua việc điều chỉnh kích thước giải đấu. Áp lực chọn lọc cao, các NST có fitness cao có nhiều khả năng được chọn hơn, có thể làm giảm sự đa dạng của quần thể.

Algorithm 2 Tournament Selection

```
population, tournament_size, population_size selected_individuals
selected_individuals ← [] each individual in population tournament_contestants ←
random selection of tournament_size individuals from population winner ← best
individual by fitness in tournament_contestants selected_individuals.append(winner)
selected_individuals
```

- **Lai ghép:** Thực hiện và so sánh giữa 3 phương pháp lai ghép: Lai ghép 1 điểm cắt, lai ghép k điểm cắt (2 điểm cắt) và lai ghép PMX.

Algorithm 3 Single Point Crossover

```
1: procedure SINGLEPOINTCROSSOVER(father1, father2, lenCities)
2:   point ← RandomInteger(0, lenCities - 1)
3:   new_invi1 ← father2[0:point]
4:   for each invi in father1 do
5:     if invi not in new_invi1 then
6:       new_invi1.append(invi)
7:     end if
8:   end for
9:   new_invi2 ← father1[0:point]
10:  for each invi in father2 do
11:    if invi not in new_invi2 then
12:      new_invi2.append(invi)
13:    end if
14:  end for
15:  return new_invi1, new_invi2
16: end procedure
```

Algorithm 4 Multi-Point Crossover

```
1: procedure MULTIPointCrossover(father1, father2, lenCities)
2:   points  $\leftarrow$  RandomSample(father1, 2)
3:   point1  $\leftarrow$  points[0] - 1
4:   point2  $\leftarrow$  points[1] - 1
5:   new_invi1  $\leftarrow$  father2[point1:point2]
6:   c1, c2  $\leftarrow$  0
7:   for each invi in father1 do
8:     if invi not in new_invi1 then
9:       c1  $\leftarrow$  c1 + 1
10:      if c1  $\leq$  point1 then
11:        new_invi1.append(invi)
12:      else
13:        new_invi1.insert(c2, invi)
14:        c2  $\leftarrow$  c2 + 1
15:      end if
16:    end if
17:  end for
18:  new_invi2  $\leftarrow$  father1[point1:point2]
19:  c1, c2  $\leftarrow$  0
20:  for each invi in father2 do
21:    if invi not in new_invi2 then
22:      c1  $\leftarrow$  c1 + 1
23:      if c1  $\leq$  point1 then
24:        new_invi2.append(invi)
25:      else
26:        new_invi2.insert(c2, invi)
27:        c2  $\leftarrow$  c2 + 1
28:      end if
29:    end if
30:  end for
31:  return new_invi1, new_invi2
32: end procedure
```

Algorithm 5 PMX (Partially Mapped Crossover)

```
1: procedure PMX(parent1, parent2)
2:   size  $\leftarrow$  length(parent1)
3:   child1, child2  $\leftarrow$  [-1] * size
4:   cx_point1, cx_point2  $\leftarrow$  RandomSample(range(size), 2)
5:   Sort(cx_point1, cx_point2)
6:   child1[cx_point1:cx_point2]  $\leftarrow$  parent1[cx_point1:cx_point2]
7:   child2[cx_point1:cx_point2]  $\leftarrow$  parent2[cx_point1:cx_point2]
8:   procedure FILLCHILD(child, parent)
9:     for ind = cx_point1 to cx_point2 do
10:      if parent[ind] not in child then
11:        pos  $\leftarrow$  ind
12:        while cx_point1  $\leq$  pos < cx_point2 do
13:          pos  $\leftarrow$  index of child[pos] in parent
14:        end while
15:        child[pos]  $\leftarrow$  parent[ind]
16:      end if
17:    end for
18:  end procedure
19:  FILLCHILD(child1, parent2)
20:  FILLCHILD(child2, parent1)
21:  for ind = 0 to size do
22:    if child1[ind] = -1 then
23:      child1[ind]  $\leftarrow$  parent2[ind]
24:    end if
25:    if child2[ind] = -1 then
26:      child2[ind]  $\leftarrow$  parent1[ind]
27:    end if
28:  end for
29:  return child1, child2
30: end procedure
```

- **Đột biến:** Thực hiện hoán đổi vị trí của 2 gen (thành phố) bất kì trong NST cha để tạo NST mới. Điều này nhằm tạo ra sự thay đổi ngẫu nhiên trong quần thể, giúp khám phá không gian tìm kiếm rộng hơn từ đó giảm khả năng mắc kẹt tại cực trị địa phương.

Bước 5: Khởi tạo hàm GA.

Đầu vào của hàm gồm các thông số: danh sách các thành phố cần tối ưu hóa (cities), quần thể ban đầu (population), số lượng thành phố, kích thước quần thể, số thế hệ (điều kiện dừng thuật toán) - cố định là 30, tỷ lệ lai ghép, tỷ lệ đột biến, hàm lai ghép (bao gồm lai ghép 1 điểm cắt, lai ghép nhiều điểm cắt và lai ghép PMX), và số lượng cá thể

được chọn ngẫu nhiên để tìm cha mẹ tốt nhất (k).

Thuật toán khởi đầu bằng việc xác định cá thể tốt nhất (elite) từ quần thể và tính toán ma trận khoảng cách giữa các thành phố. Trong mỗi thế hệ, một danh sách cá thể mới (new_individuals) được tạo ra. Đối với mỗi cặp cá thể, nếu giá trị ngẫu nhiên nhỏ hơn crossover_rate, hai cá thể cha mẹ tốt nhất sẽ được chọn từ k cá thể ngẫu nhiên, sau đó lai ghép bằng hàm crossover_func để tạo hai cá thể con. Nếu giá trị ngẫu nhiên nhỏ hơn mutation_rate, các cá thể con sẽ được đột biến. Nếu không xảy ra lai ghép, hai cá thể mới sẽ được tạo ngẫu nhiên. Tất cả cá thể con được thêm vào danh sách new_individuals. Sau đó, quần thể được cập nhật thông qua chọn lọc giải đấu, và cá thể tốt nhất (elite) được cập nhật nếu cần.

Algorithm 6 Genetic Algorithm (GA)

```
cities, population, lenCities, population_size, num_of_generations,
crossover_rate, mutation_rate, crossover_func, k sorted population, elite
elite ← best individual in population
distance_mat ← distance_matrix(cities)
each generation
new_individuals ← []
1 to population_size / 2
  random ≤ crossover_rate
  father1 ← best of  $k$  random individuals
  father2 ← best of  $k$  random individuals
  inv1, inv2 ← crossover_func(father1, father2)
  random ≤ mutation_rate
  inv1 ← mutation(inv1)
  inv2 ← mutation(inv2)
  inv1, inv2 ← random individuals
  add inv1 and inv2 to new_individuals
population ← tournaments(population, new_individuals)
update elite with best individual
sorted population, elite
```

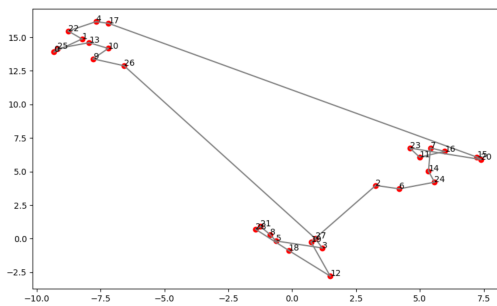
4 Đánh giá và kết luận

Sau khi thực nghiệm thuật toán với 4 datasets cỡ từ nhỏ đến lớn, thu được kết quả và đánh giá qua bảng sau:

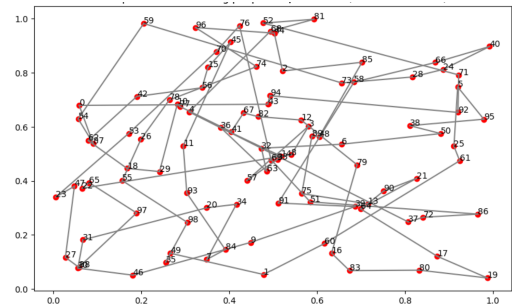
Data	Tiny	Small	Medium	Large
Số thành phố	10	30	100	1000
Số lần lặp	500	500	500	500
Thời gian sử dụng lai ghép 1 điểm cắt	0.84	1.71	6.8	341.24
Thời gian sử dụng lai ghép 2 điểm cắt	0.66	1.34	5.64	187.6
Thời gian sử dụng lai ghép PMX	0.65	1.21	3.95	13.46
Độ thích nghi lớn nhất	0.08726	0.01913	0.05302	0.0023
Độ dài chu trình ngắn nhất	11.46	52.28	18.86	358.31

Bảng 1: Kết quả thực nghiệm

Có thể thấy, khi tăng tích cỡ bộ dữ liệu (số lượng các thành phố) lên nhiều lần, thời gian tính toán và độ phức tạp của mỗi vòng lặp tăng lên đáng kể. Vì vậy, với những dataset cỡ chục nghìn (10000 - 20000 thành phố) vẫn đang là bài toán khó và cần kết hợp với các thuật toán khác để giảm thời gian và khối lượng tính toán. Sau 500 thế hệ (vòng lặp), ta thu được các đường đi có độ dài ngắn nhất (NST có hàm thích nghi cao nhất) như sau:



(a) Data small



(b) Data medium

Hình 1: Biểu đồ đường đi tối ưu cho bộ dữ liệu nhỏ và vừa

4.1 So sánh hiệu quả giữa các phép lai ghép

Khi thực nghiệm giải thuật di truyền với phép lai ghép 1 điểm cắt (Single point crossover), thời gian thực thi thường lâu hơn các phép lai ghép khác. Lai ghép 1 điểm cắt hoạt động như sau: chọn một điểm chia cố định trên chuỗi gen để hoán đổi đoạn phía sau giữa hai cá thể cha mẹ. Thao tác này có thể dẫn đến việc sinh ra nhiều cá thể không hợp lệ (ví dụ: chứa gen trùng lặp trong bài toán TSP). Khi xảy ra cá thể không hợp lệ, thuật toán cần thêm bước sửa lỗi hoặc loại bỏ chúng, làm tăng thời gian xử lý. Ngoài ra, lai ghép 1 điểm cắt thường duy trì tính tương đồng cao giữa các cá thể con và cha mẹ, dẫn đến quần thể tiến hóa chậm hơn và mất thêm thời gian để đạt được giải pháp tốt hơn.

Với lai ghép nhiều điểm cắt, chọn nhiều điểm chia và trao đổi đoạn gen xen kẽ giữa các cá thể cha mẹ. Phép lai ghép này phân phối gen từ cha mẹ đều hơn, thường tạo ra cá thể con hợp lệ với xác suất cao hơn, giảm bớt thời gian sửa lỗi. Lai ghép nhiều điểm cắt cũng tạo ra các cá thể con có sự pha trộn tốt hơn giữa gen của hai cha mẹ, tăng cơ hội tìm kiếm giải pháp tốt nhanh hơn, do đó thời gian chạy tổng thể ngắn hơn so với lai ghép 1 điểm cắt.

Với PMX, cấu trúc hoán vị được duy trì trực tiếp trong quá trình lai ghép: Các phần tử được trao đổi một cách có kiểm soát giữa hai cha mẹ, các thành phố bị trùng lặp sẽ

được ánh xạ (mapping) lại theo quy tắc của PMX, đảm bảo rằng cá thể con luôn hợp lệ ngay từ đầu. Việc này giúp giới hạn số lượng thay đổi cần thực hiện, giảm thời gian xử lý, trong khi lai ghép 1 điểm cắt và lai ghép nhiều điểm cắt có xu hướng tạo ra các cá thể không hợp lệ (các thành phố bị lặp lại hoặc thiếu), cần các bước bổ sung để kiểm tra và sửa chữa, tăng thời gian chạy. Ngoài ra, PMX thường duy trì được nhiều đặc điểm tốt của cha mẹ trong khu vực hoán vị được lai ghép. Điều này giúp thuật toán hội tụ nhanh hơn, giảm số thế hệ cần thiết.

4.2 Phương án cải thiện đề xuất: kết hợp với tìm kiếm Tabu

Thuật toán tìm kiếm địa phương bắt đầu bằng cách đưa ra một lời giải hợp lệ ban đầu cho bài toán, sau đó xem xét các lời giải láng giềng để tìm kiếm một giải pháp tốt hơn. Tuy nhiên, phương pháp này thường gặp phải hai vấn đề: nó dễ bị mắc kẹt ở các điểm tối ưu cục bộ và có thể lặp đi lặp lại giữa các lời giải láng giềng giống nhau mà không tiến triển. Tabu Search cải thiện hiệu quả của tìm kiếm địa phương bằng cách đưa ra các quy tắc linh hoạt hơn:

- Nếu không tìm được lời giải láng giềng tốt hơn lời giải hiện tại, thuật toán cho phép chấp nhận tạm thời một lời giải kém hơn. Điều này giúp vượt qua các điểm tối ưu cục bộ.
- Để tránh lặp lại các lời giải đã từng thử trong thời gian gần, thuật toán sử dụng một "danh sách cấm" (tabu list). Những lời giải trong danh sách này sẽ bị chặn, không cho phép di chuyển đến chúng từ lời giải hiện tại.

Nhờ đó, Tabu Search có thể khám phá các vùng không gian tìm kiếm rộng hơn và tăng cơ hội tìm ra lời giải tối ưu tốt hơn. Một số phương án kết hợp GA và Tabu:

- Áp dụng Tabu Search cho mỗi cá thể trong quần thể của GA. Khi một cá thể được chọn để lai ghép hoặc đột biến, chạy Tabu Search trên cá thể đó để cải thiện giải pháp trước khi đưa nó vào thế hệ mới.
- Sử dụng Tabu Search để tối ưu hóa quần thể GA trong suốt quá trình tìm kiếm, bao gồm áp dụng Tabu Search cho các giai đoạn tìm kiếm quần thể, giúp điều chỉnh động học quần thể hoặc kiểm soát việc chọn lọc. Sau một số thế hệ nhất định, sử

dùng Tabu Search để điều chỉnh lại các cá thể trong quần thể, có thể loại bỏ những giải pháp đã tìm kiếm gần đây hoặc tránh bị mắc kẹt trong cực trị cục bộ.

Tài liệu

- [1] Sourabh Katoch, Sumit Singh Chauhan **and** Vijay Kumar. “A review on genetic algorithm: past, present, and future”. **in***Multimedia Tools and Applications*: 79.9 (2020), **pages** 11225–11245. DOI: [10.1007/s11042-020-10139-6](https://doi.org/10.1007/s11042-020-10139-6).
- [2] Melanie Mitchell. *An Introduction to Genetic Algorithms*. Fifth printing, 1999 (First MIT Press paperback edition, 1998). Includes bibliographical references and index. Cambridge, Massachusetts, London, England: A Bradford Book, The MIT Press, 1996, **pages** 8, 8, 117–118, 128–130. ISBN: 0-262-13316-4 (HB), 0-262-63185-7 (PB).
- [3] Matias T. M. Merich **and** Art J. Deutz. “A tutorial on multiobjective optimization: fundamentals and evolutionary methods”. **in***Natural Computing*: 17.3 (2018), **pages** 585–609. DOI: [10.1007/s11047-018-9723-5](https://doi.org/10.1007/s11047-018-9723-5).
- [4] B. Freisleben **and** P. Merz. “New Genetic Local Search Operators for the Traveling Salesman Problem”. **in***Proceedings of the Fourth Conference on Parallel Problem Solving from Nature*: **by editor**H.-M. Voigt **and others**. **volume** 1141. Lecture Notes in Computer Science. Berlin, Germany: Springer-Verlag, 1996, **pages** 890–899.
- [5] Saloni Gupta **and** Dr. Panwar. “Solving Travelling Salesman Problem Using Genetic Algorithm”. **in***International Journal of Advanced Research in Computer Science and Software Engineering*: 3 (2013), **pages** 376–380.
- [6] Philip Adewole, Adio Akinwale **and** Kehinde Otubamowo. “A Genetic Algorithm for Solving Travelling Salesman Problem”. **in***International Journal of Advanced Computer Sciences and Applications*: 2 (2011). DOI: [10.14569/IJACSA.2011.020104](https://doi.org/10.14569/IJACSA.2011.020104).
- [7] I. Smith. *Overview Evolutionary Algorithms*. ScienceDirect. 2011. DOI: [10.1016/j.knosys.2011.04.003](https://doi.org/10.1016/j.knosys.2011.04.003).